

HiFloat8浮点数据格式：既要又要之路

介绍HiFloat8（HiF8）之前，我们先回顾一下AI浮点数据格式的发展，相应的基本概念，并作简单的对比分析。以便读者更容易理解这篇技术分享的内容。

1. 传统16位浮点数

计算机用IEEE 754标准存储数据，其核心类似于十进制的科学计数法，例如 -1.25×10^{-3} ，只不过是二进制版本。以半精度FP16浮点为例，它用16比特（2字节）存储一个数。这16位被划分为3块：

- 符号位（Sign bit）：1 bit，1表述负，0表示正
- 阶码位（Exponent Field）：5 bits，编码 $[0, 31] - 15 = [-15, 16]$ ，其中15是阶码偏置。E = -15，表示非常规数值（Subnormal/Denormal），E = 16时编码无穷和NaN，E等于其他值时表示常规数值。
- 尾数位（Mantissa Field）：10 bits，都表示小数位，小数位之前有1-bit值固定的隐藏位（Normal模式是1，Subnormal模式是0），共11比特有效位

因此FP16的二进制科学计数法表达式如下（阶码E已经过偏置还原）：

$$\text{Normal: } X = (-1)^S \times 2^E \times 1.M_2$$

$$\text{Subnormal: } X = (-1)^S \times 2^{-14} \times 0.M_2$$

浮点数相比定点数，通过二进制的阶码和尾数分离的科学计数法结构，在有限的硬件位宽下，实现了巨大的动态范围。比如INT16，最多只能表示 $\pm[2^0, 2^{15} - 1]$ 范围的非0数值，等效支持的二进制指数为 $[0, 14]$ ，共15个；而FP16却可以表达 $\pm[2^{-24}, 2^{15} \times (2 - 2^{-10})]$ 这么大范围的非0数值，等效支持的二进制指数高达 $[-24, 15]$ ，共40个。

FP16在CNN流行的时代，是主流的训练数据格式。但是随着LLM的爆发和流行，人们逐渐发现语言模型训练过程中的数值范围非常弥散，仅有40个指数支持的FP16经常有大量数值下溢变成0，影响模型训练精度甚至直接跑飞（即便已经支持了反向全局Loss Scaling）。因此谷歌早期提出的BF16数据格式逐渐替代FP16，成为了LLM时代的主流训练格式。BF16数据格式具有1-bit符号位，8-bit阶码和，7-bit尾数，等价于直接把FP32数据格式的低16-bit尾数直接砍掉。相比于FP16，BF16虽然只有8位有效位，但却能支持 $[-133, 127]$ 的超大二进制指数范围（包括Subnormal），对LLM训练稳定性和精度收敛极其友好。

2. FP8浮点数

FP8通常包括E4M3和E5M2两种（默认最高位存在1 bit符号位），直接继承了IEEE 754浮点数的特征，使用固定位宽的阶码域和尾数域表达数值大小。其中E4M3设定4-bit阶码和3-bit尾数，精度相比E5M2较高，有4位有效位，但是动态范围很窄，只能支持 $[-9, 8]$ ，共18个二进制指数表达（包括Subnormal）；而E5M2设定了5-bit阶码和2-bit尾数，精度相对E4M3较低，只有3位有效位，但是动态范围相对较大，能支持 $[-16, 15]$ ，共32个二进制指数表达（包括Subnormal）。通常E4M3用于神经网络前向的激活值A和权重W，E5M2用于神经网络后向的激活值梯度dA。但近期业界认为E5M2精度过低，像DeepSeek_V3在训练的时候，dA也用了E4M3。

3. 现有浮点格式分析

图1展示了FP16, BF16和FP8的域位宽结构。他们都包括三个域，符合IEEE 754的基本特征。其中阶码域决定基础动态范围，尾数域决定精度（有效位）。同时较宽的尾数域在Subnormal模式下能补充支持一定的二进制指数值，扩大一些动态范围。

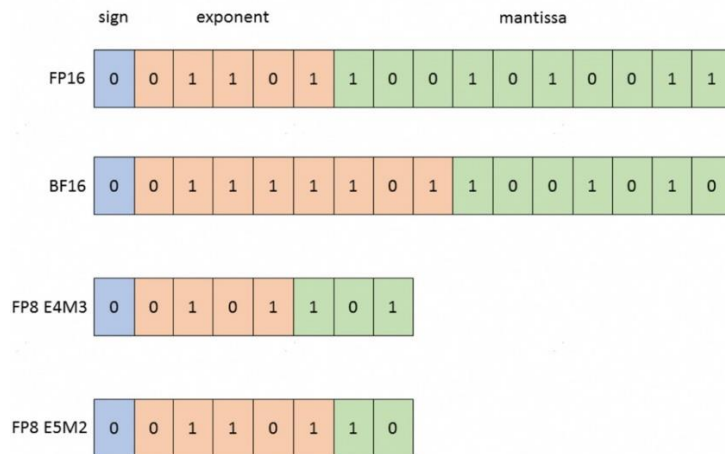


图1. Float16和Float8域位宽结构示意图

如前所述，具有40个二进制指数支持的FP16在LLM训练竞争中，已经逐渐败给了具备超大动态范围的BF16格式。毕竟现在大模型的训练成本太高了，训练不稳定的损失，很多企业和机构都难以承受。而现有的FP8-E4M3，最多只能支持18个二进制指数，远远落后于FP16。这使得FP8训练的稳定性面临极大的挑战。为了解决这一问题，极端细粒度scaling和current scaling（FP8 delayed scaling训练因为极其不稳定，尚未看见成功案例），成为了稳定训练的常见手段。但这些技术方案会明显降低8位浮点的训练加速比。比如Ling-1T是目前已知最大的FP8训练的基础模型，但其FP8混合精度训练仅仅只带来了15%+的端到端加速，远远低于预期。毕竟8-bit浮点的矩阵乘算力，相对16-bit浮点是翻倍了的。

综上，LLM训练对动态范围的诉求极大，导致FP16逐渐式微于BF16。而FP8-E4M3的动态范围比FP16的一半还要小，这使得大家只能牺牲大量的训练性能提升，来换取训练稳定性和精度收敛。

4. HiF8浮点数

FP8困境的根源在于，对于LLM来说，精度很重要，动态范围也很重要。在8-bit位宽限制下，简单沿用传统IEEE 754数据格式的技术方案，无法做到“既要又要”。为此，我们提出了锥形精度的浮点数据格式HiF8。

如图2所示，HiF8在传统的符号域，阶码域，和尾数域的基础上，引入了即时可译的变长前缀码编码的点位域Dot，直接显示指示阶码存储的位宽，和Denormal标志。Dot分别编码了0~4五个值，因此阶码位宽最少0 bit，最多4 bits。在已知符号位，点位域，和阶码域位宽的基础上，并且格式总共只有8-bit，尾数位宽可对应求出。

为了确保不同位宽的阶码域，所表达的指数值不重复，实现无冗余编码，HiF8采用Sign-Magnitude的原码编码阶码域，并且固定幅值的最高位为确定值，不存储。比如说，当Dot = 0时，说明阶码不占据位宽，并且E直接等于0；而当Dot = 1~4时，固定阶码幅值的最高位为1（示意图中的红色数字），不占据存储位宽。基于这种规则，Dot

= 1, 阶码E只能等于±1, 不包含已经编码的0; Dot = 2时, 阶码E只能等于±[2,3], 不包含已经编码的[-1, 1]。以此类推, Dot域指示的0~4五种位宽的阶码, 共编码了[-15, 15]的指数, 且彼此之间不重复, 实现了无冗余编码。



图2. HiF8编码示意图

同时, 我们特意用大位宽的Dot域指示小位宽并且小幅值的阶码域, 实现了尾数精度的渐变, 而不是跳变。可以看到, 在E = [-3, 3]时, 有3-bit尾数。然后随着阶码幅值变大, Normal模式下的尾数位宽逐渐减少到1-bit。这是HiF8的老师, Posit数据格式所不具备的。

Normal模式下, HiF8的数值解析式和传统IEEE 754格式一样, 是标准的二进制科学计数法表示。为了支持更大的动态范围, HiF8还采用了不同于IEEE 754风格的Subnormal/Denormal方案。如图2的Denormal表达式, 当Dot域指示该数值为Denormal模式时, HiF8没有指数域, 多的3-bit“尾数”(编码0~7八个数值), 直接用于扩展HiF8的小值指数范围。HiF8 Denormal模式在Normal模式支持的[-15, 15]共31个指数的基础上, 额外扩展了[-22, -16]共7个指数(-23用于表示特殊值Zero和NaN), 综合形成了[-22, 15]共38个指数的大动态范围, 非常接近FP16的[-24, 15]的40个指数范围。并且由于[-15, -8]本身是1-bit尾数, 扩展的[-22, -16]的0-bit尾数与之相邻, 也属于精度渐变, 并非跳变。

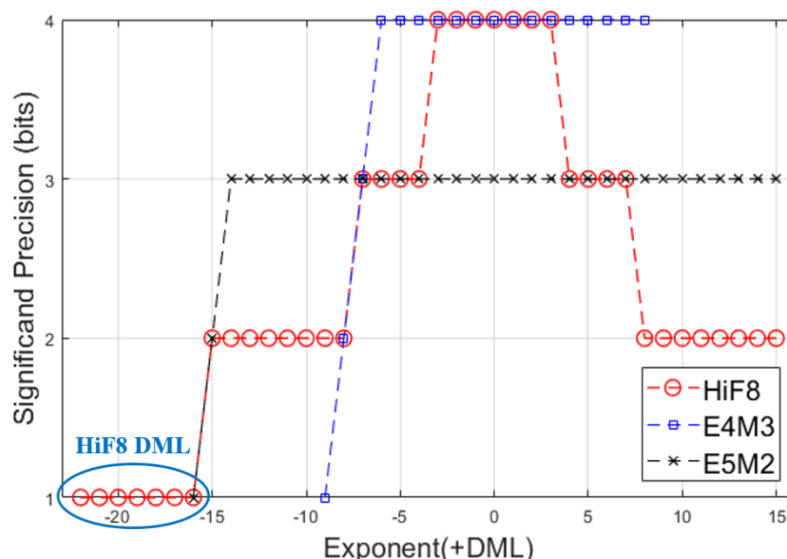


图3. Float8有效位-指数示意图

根据前述HiF8编码描述, 和FP8的回顾, 可以画出图3所示的Float8有效位-指数示意图。其中HiF8精度明显不

同于域位宽固定的E4M3和E5M2，具备一种锥形精度的特征。锥形精度的概念启发于Posit数据格式——工程计算中的绝大多数数值分布，都具备类高斯分布的聚合特征。神经网络的训练和推理也不例外。只要众数数值能用高精度充分表达即可，两边占比较小的数据（幅值较大或者较小），精度可以逐渐降低一些，不会显著影响端端的功能和效果。但Posit的编码方式，尾数精度在指数变化时存在严重跳变，精度分配不够均衡。在8-bit限制下，Posit无法很好匹配当前神经网络的诉求。HiF8作为Posit的学生，希望能将锥形精度的思想发扬光大。

When $S = 0$, $D = \text{DML}$, $M = 3'b0$, $X = \text{Zero}$ (HiF8: 8'b00000000)
 When $S = 1$, $D = \text{DML}$, $M = 3'b0$, $X = \text{NaN}$ (HiF8: 8'b10000000)
 When $D = 4$, $E_s = 4'b0111 = 15$, $M = 1'b1$ 时, $X = \pm \text{Inf}$ (HiF8: 8'bX1101111)

图4. HiF8特殊值支持

除了常规数值编码，HiF8也编码了4个特殊值，具体如图4所示。可以看到，HiF8编码了Zero，NaN和正负无穷。其中HiF8不区分正0和负0，或者可以理解为HiF8用单一pattern同时表示了 ± 0 （神经网络区分正负0意义不大，8-bit表达空间十分有限，浪费不划算）。因此，HiF8是一个数值表达能力完备的8-bit单数据格式。

5. 总结和展望

表格1详细总结对比了HiF8，FP8和FP16的典型值和关键特征。可以看到，HiF8利用匹配数据分布的锥形精度特征，成功在8-bit限制下，在保证神经网络需求精度的前提下，显著扩大了格式的动态范围。进而为神经网络训练和推理，提供了能力更全面的8-bit单数据格式表达。

表1. HiF8，FP8和FP16的典型值和关键特征

	FP16	HiF8	FP8-E4M3	FP8-E5M2
Infinites	Support	$S1101111_2$	N/A	Support
NaN	Support	10000000_2	Support	Support
Zero	Support	00000000_2	Support	Support
Max Positive NML	$2^{15} \times (2 - 2^{-10})$	$01101110_2 = 2^{15}$	1.75×2^8	1.75×2^{15}
Min Positive NML	2^{-14}	$01111110_2 = 2^{-15}$	2^{-6}	2^{-14}
Max Positive DML	$2^{-14} \times (1 - 2^{-10})$	$00000111_2 = 2^{-16}$	1.75×2^{-7}	1.5×2^{-15}
Min Positive DML	2^{-24}	$00000001_2 = 2^{-22}$	2^{-9}	2^{-16}
Exponent	$[-14, 15]$	$[-15, 15]$	$[-6, 8]$	$[-14, 15]$
Exponent (+DML)	$[-24, 15]$	$[-22, 15]$	$[-9, 8]$	$[-16, 15]$

由于FP16在和BF16的LLM训练竞争中，40个指数的动态范围都日渐式微（LLM训练太吃动态范围了！）。HiF8的38个指数表达显然还是不够的。但是相比于FP8，尤其是E4M3的18个指数表达，动态范围已经是成倍提升。我们相信这种优势最终会转换到神经网络E2E性能或者精度上。在8-bit的极低开销限制下，HiF8已经算是成功找到了一条“既要有要”的道路。后续我们会继续发掘并向大家呈现HiF8在神经网络训练和推理中的优势。